Projeto: Video Poker

Neste projeto você vai implementar o **Video Poker**, a máquina de poker dos cassinos, em Python, aplicando os conceitos de orientação a objetos: encapsulamento, propriedades, herança, classes abstratas, enumerações, sobrecarga de operadores e exceções.

O projeto é dividido em duas partes:

1. **Modelo do jogo** (`card.py`, `card_collections.py`, `poker.py`, `exceptions.py`): toda a lógica e as regras do jogo, sem nenhum `print`, `input` ou código de interface.
2. **Interfaces** (`poker_terminal.py` e `poker_gui.py`): mostram as cartas e a pontuação e repassam as escolhas do jogador para o modelo.

Como o modelo não depende de interface, as duas interfaces usam exatamente a mesma classe `VideoPoker`.

As classes e métodos apresentados neste documento são apenas sugestões, você pode criar a estrutura que achar melhor. A única exigência é que a lógica e as regras do jogo fiquem separadas da interface.

Regras do jogo

A versão mais comum do video poker se chama **Jacks or Better**. Uma rodada funciona assim:

- **Distribuição (deal):** a máquina dá **5 cartas** de um baralho completo de 52 cartas, embaralhado a cada rodada.
- **Escolha (hold):** o jogador marca quais cartas quer **manter**, de nenhuma a todas.
- **Troca (draw):** as cartas não marcadas são substituídas por novas, tiradas do **mesmo baralho**. Só há **uma troca** por rodada.
- **Pontuação:** a mão final é avaliada e o jogador ganha os pontos da tabela abaixo, que se somam ao total acumulado.

Mão final	Descrição	Pontos
Royal flush	Sequência de 10 a ás, toda do mesmo naipe	250
Straight flush	Sequência toda do mesmo naipe	50
Quadra	4 cartas do mesmo valor	25
Full house	Uma trinca e um par	9
Flush	5 cartas do mesmo naipe	6
Sequência	5 valores consecutivos	4
Trinca	3 cartas do mesmo valor	3
Dois pares	Dois pares de valores diferentes	2
Jacks or better	Um par de valetes, damas, reis ou ases	1

Mão final	Descrição	Pontos
Nada	Qualquer outra mão	0

O nome "Jacks or Better" vem da penúltima linha: um par só pontua se for de **valetes ou maior**. Um par de 10 ou menor não vale nada.

Na sequência, o ás pode ser a carta mais alta (10-J-Q-K-A) ou a mais baixa (A-2-3-4-5). Fora isso, o ás é sempre a carta mais alta.

Avaliando a mão

Quase todas as mãos são reconhecidas contando **quantas cartas há de cada valor** e ordenando essas contagens da maior para a menor:

Contagens	Mão
`[4, 1]`	Quadra
`[3, 2]`	Full house
`[3, 1, 1]`	Trinca
`[2, 2, 1]`	Dois pares
`[2, 1, 1, 1]`	Um par (pontua só se for de valete ou maior)
`[1, 1, 1, 1, 1]`	Nada, sequência ou flush

Exemplo: a mão `K♠ K♥ 7♦ K♣ 7♠` tem três reis e dois setes, então as contagens são `[3, 2]`, um full house.

Além das contagens, você precisa de mais duas informações:

- **Flush:** todas as cartas têm o mesmo naipe, ou seja, o **conjunto** de naipes tem tamanho 1.
- **Sequência:** os 5 valores são diferentes e a diferença entre o maior e o menor é 4. O caso especial é A-2-3-4-5, com os números `[2, 3, 4, 5, 14]`.

As verificações precisam ser feitas **da mão mais forte para a mais fraca**, porque uma mão forte também satisfaz condições de mãos mais fracas. Um straight flush, por exemplo, também é um flush e também é uma sequência.

> Dica: `collections.Counter` conta as ocorrências de cada valor em uma linha: `Counter(card.rank for card in hand)`.

Cartas em Unicode

O Unicode tem um caractere para cada carta do baralho, no bloco que começa em `U+1F0A0`. Cada naipe ocupa uma faixa de 16 posições, e a posição da carta dentro da faixa é o valor dela:

Naipe	Base	Ás	2 … 10	Valete	(Cavaleiro)	Dama	Rei
♠ Espadas	`0x1F0A0`	`+0x1`	`+0x2` … `+0xA`	`+0xB`	`+0xC`	`+0xD`	`+0xE`
♥ Copas	`0x1F0B0`						
♦ Ouros	`0x1F0C0`						
♣ Paus	`0x1F0D0`						

Repare que existe uma carta a mais, o **Cavaleiro** (`+0xC`), que não faz parte do baralho comum. Por isso a dama e o rei ficam uma posição depois do que se esperaria. O verso da carta é o próprio `U+1F0A0`.

Exemplo: o rei de copas é `chr(0x1F0B0 + 0xE)`, que mostra 🂾.

Classes

Exceções (`exceptions.py`)

Classe	Responsabilidade
`GameError(Exception)`	Exceção base de todos os erros do jogo.
`EmptyStockError(GameError)`	Lançada quando se tenta tirar uma carta de um baralho vazio.
`CardNotInHandError(GameError)`	Lançada quando se tenta descartar ou trocar uma carta que não está na mão.
`InvalidActionError(GameError)`	Lançada quando uma ação não é permitida na fase atual da rodada.
`InvalidPositionError(InvalidActionError)`	Lançada quando a posição de uma carta a manter está fora do intervalo 0 a 4.

`Suit` e `Rank` (`card.py`)

Enumerações (`Enum`) com os naipes e os valores das cartas. Cada membro guarda uma tupla de dados, que é desempacotada no `__init__`:

- `Suit`: `CLUBS`, `DIAMONDS`, `HEARTS` e `SPADES`. Cada naipe tem `order` (para ordenar), `symbol` (`'♣'`, `'♦'`, `'♥'`, `'♠'`) e `unicode_base` (a base da tabela acima).
- `Rank`: de `TWO` a `ACE`. Cada valor tem `number` (de 2 a 14, com o ás valendo 14) e `label` (`'2'` … `'10'`, `'J'`, `'Q'`, `'K'`, `'A'`).

As duas enumerações implementam `__lt__`, para que seja possível comparar, por exemplo, `Rank.QUEEN > Rank.JACK`.

> Dica: o decorador `functools.total_ordering` gera `<=`, `>` e `>=` automaticamente a partir de `__eq__` e `__lt__`.

`Card` (`card.py`)

Representa **uma carta** do baralho, que pode estar virada para cima ou para baixo.

- `__init__(rank: Rank, suit: Suit, face_up: bool = True)`.
- Propriedades somente leitura `rank`, `suit` e `is_face_up`.
- `flip()`: vira a carta.
- `__eq__` e `__hash__`: duas cartas são iguais se têm o mesmo valor e o mesmo naipe, mesmo que uma esteja virada e a outra não.
- `__lt__`: compara primeiro pelo valor e, em caso de empate, pelo naipe.
- `__str__()`: o caractere Unicode da carta, ou o verso (`BACK_UNICODE`) se ela estiver virada para baixo.
- `__repr__()`: texto legível para depuração, como `Card(K♥)`.

`CardCollection` (`card_collections.py`), classe abstrata

Base para qualquer conjunto de cartas. Guarda as cartas em uma lista protegida `_cards`.

- `__init__(cards: Iterable[Card] = ())`.
- `__len__()`: quantidade de cartas.
- `__iter__()`: **método abstrato**. Cada subclasse define em que ordem as cartas são percorridas.

`Stock` (`card_collections.py`)

O **baralho** de onde as cartas são tiradas. O topo do baralho é o **fim** da lista.

- `__init__()`: cria as 52 cartas, todas viradas para baixo.
- `shuffle()`: embaralha as cartas (use `random.shuffle`).
- `draw() -> Card`: retira e retorna a carta do topo. Lança `EmptyStockError` se o baralho estiver vazio.
- `__iter__()`: percorre do topo para o fundo.

`Hand` (`card_collections.py`)

As **cartas na mão** do jogador, na ordem em que estão na mesa.

- `receive(card)`: acrescenta uma carta no fim da mão.
- `discard(card) -> Card`: remove e retorna a carta. Lança `CardNotInHandError` se ela não estiver na mão.
- `replace(old, new) -> Card`: troca `old` por `new` **na mesma posição** e retorna `old`. Lança `CardNotInHandError` se `old` não estiver na mão.
- `__iter__()`: percorre na ordem da mesa.

`HandType` e `Phase` (`poker.py`)

- `HandType`: enumeração com as mãos da tabela de pontuação. O **valor** de cada membro é a pontuação da mão (`NOTHING = 0`, `JACKS_OR_BETTER = 1`, …, `ROYAL_FLUSH = 250`). Também implementa `__lt__`, comparando pela pontuação.
- `Phase`: as fases de uma rodada.
  - `READY`: esperando uma nova rodada. É a fase inicial e também a fase depois da troca.
  - `HOLDING`: há 5 cartas na mesa e o jogador está escolhendo quais manter.

`VideoPoker` (`poker.py`)

É o **modelo do jogo**: controla o baralho, a mão, a fase da rodada e a pontuação. As interfaces (terminal ou gui) interagem com essa classe.

Estado interno:

- `_stock`: o baralho da rodada atual.
- `_hand`: as cartas na mesa.
- `_phase`: a fase atual.
- `_current_points`: os pontos ganhos na última rodada.
- `_total_points`: os pontos acumulados desde o começo do jogo.

Interface pública:

- `HAND_SIZE = 5`: constante de classe com o número de cartas na mão.
- `phase -> Phase`: a fase atual.
- `points -> int`: os pontos acumulados.
- `current_points -> int`: os pontos da última rodada.
- `cards -> tuple[Card, ...]`: as cartas na mesa, como tupla, para que a interface não consiga alterar a mão.
- `deal()`: começa uma rodada. Cria um baralho novo, embaralha, tira 5 cartas, vira cada uma para cima e passa para a fase `HOLDING`. Lança `InvalidActionError` se a fase não for `READY`.
- `draw(held: Iterable[int])`: recebe as **posições** (0 a 4) das cartas a manter e troca todas as outras por cartas novas do baralho. Depois avalia a mão, soma os pontos e volta para a fase `READY`. Lança `InvalidActionError` se a fase não for `HOLDING` e `InvalidPositionError` se alguma posição estiver fora de 0 a 4.
- `hand_type() -> HandType`: avalia as cartas na mesa, como descrito em Avaliando a mão.

Depois do `draw`, as cartas continuam na mesa até o próximo `deal`, para que a interface possa mostrar a mão final.

> Dica: para validar as posições, transforme `held` em `set` e verifique se ele é subconjunto de `set(range(HAND_SIZE))` com o operador `<=`.

Interfaces

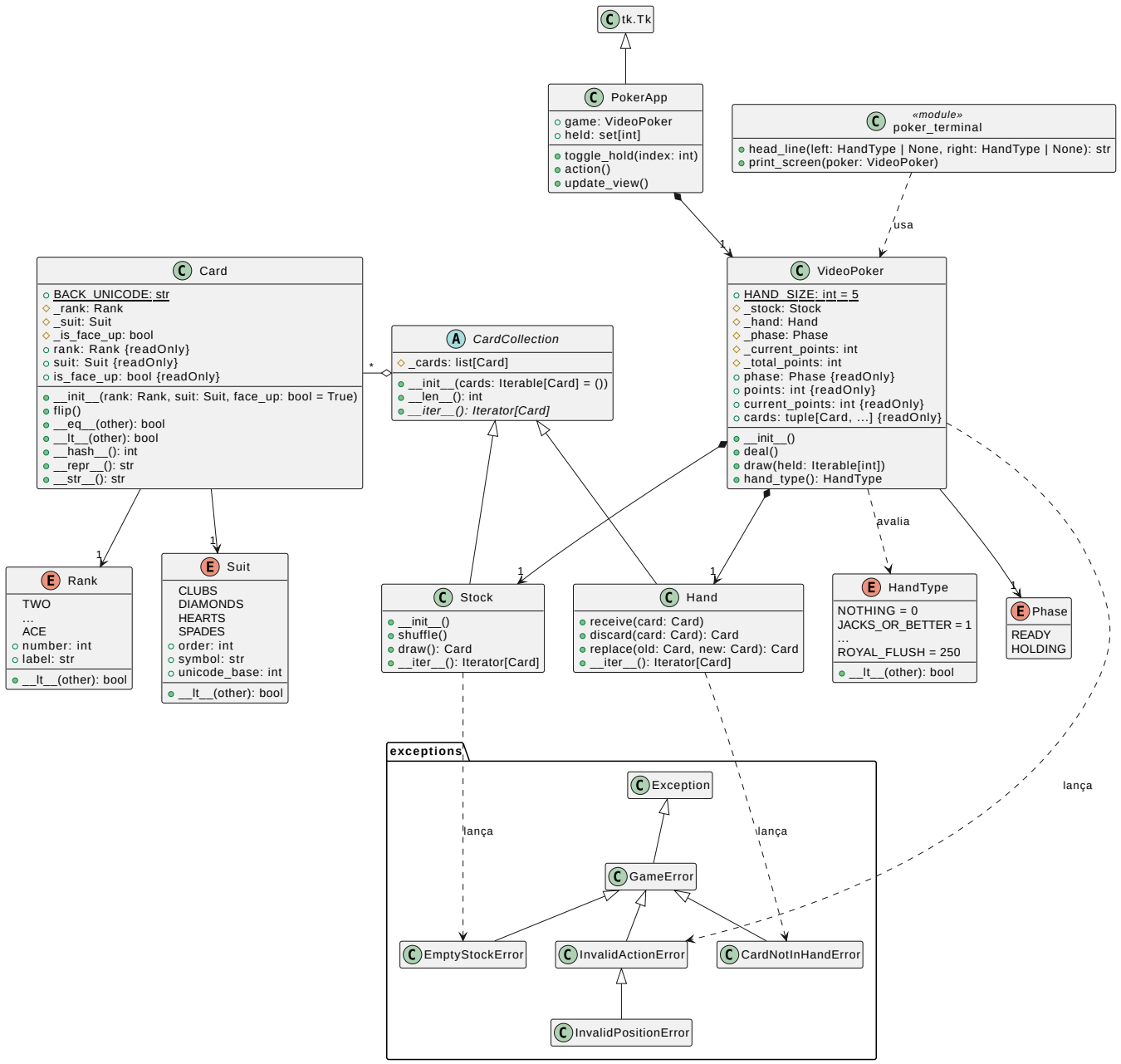
As interfaces **não implementam regra nenhuma**. Elas só mostram o estado do jogo (usando `cards`, `phase`, `points`, `current_points` e `hand_type()`) e chamam `deal()` e `draw()`.

As duas interfaces mostram a tabela de pontuação e usam um dicionário (`HAND_NAMES`) para traduzir cada `HandType` no nome que aparece na tela.

`poker_terminal.py`: mostra a tabela de pontuação, as 5 cartas com a posição de cada uma (1 a 5) embaixo, o nome da mão final e os pontos acumulados. Na fase `READY`, o jogador aperta **Enter** para dar as cartas. Na fase `HOLDING`, digita as posições das cartas que quer manter, separadas por espaço (por exemplo `1 3 4`). Uma linha vazia troca todas. Se o jogador digitar uma posição inválida, a interface mostra um aviso e pergunta de novo.

`poker_gui.py`: classe `PokerApp`, que herda de `tk.Tk`. Mostra a tabela de pontuação, as cartas grandes (cartas de copas e ouros em vermelho) e um botão que alterna entre **Dar cartas** e **Trocar**. Na fase `HOLDING`, clicar em uma carta marca ou desmarca a carta para manter, e aparece **MANTER** embaixo das marcadas. Ao fim da rodada, a mão obtida fica destacada na tabela, as cartas ficam acinzentadas e aparece quantos pontos o jogador ganhou.

Diagrama de classes



Ordem sugerida de implementação

1. `exceptions.py`, `Suit`, `Rank` e `Card`. Teste com `print` para conferir os caracteres Unicode de algumas cartas, principalmente as damas e os reis.
2. `CardCollection`, `Stock` e `Hand`. Confira se o baralho tem 52 cartas diferentes e se `replace` mantém a posição da carta.

3. `HandType` e `VideoPoker.hand_type()`. Teste montando mãos conhecidas (uma para cada linha da tabela, além de A-2-3-4-5 e de um par de 10).
4. O restante de `VideoPoker`: `deal()`, `draw()` e as propriedades.
5. `poker_terminal.py`, para jogar e testar as regras.
6. `poker_gui.py`.